

KerpheX

Présentation complète du projet

Blog professionnel Laravel 13 — Documentation technique et pédagogique

Auteur : Kerphile Saint

Date : Juin 2026

Environnement local : Laragon (Windows)

Production : Railway.app

URL : <https://web-production-c5c2f.up.railway.app>

Table des matières

1. Introduction — Qu'est-ce qu'un blog et pourquoi ce projet ?
2. Technologies et outils utilisés
3. Environnement de développement — Laragon
4. Architecture globale du projet
5. Structure des dossiers — chaque fichier expliqué
6. Base de données et modèles de données
7. Routes et contrôleurs — le cœur de l'application
8. Vues Blade et interface utilisateur
9. Authentification et gestion des rôles
10. Fonctionnalités détaillées
11. Services, middleware et sécurité
12. Assets frontend — CSS, JavaScript, Vite
13. Seeders et données de démonstration
14. Tests automatisés
15. Déploiement — GitHub et Railway
16. Liens utiles et identifiants
17. Conclusion

1. Introduction — Qu'est-ce qu'un blog et pourquoi ce projet ?

1.1 L'utilité d'un blog

Un **blog** (contraction de « web log », journal en ligne) est un site web dont le contenu principal est constitué d'**articles** publiés régulièrement, classés par date, thème ou catégorie. Contrairement à un site vitrine statique, un blog est *dynamique* : son contenu évolue dans le temps.

Pourquoi créer un blog ?

- **Partager des connaissances** — Tutoriels, analyses, retours d'expérience.
- **Construire une identité professionnelle** — Portfolio éditorial, expertise visible.
- **Créer une communauté** — Commentaires, likes, favoris, échanges avec les lecteurs.
- **Améliorer le référencement (SEO)** — Contenu frais indexé par les moteurs de recherche.
- **Centraliser ses idées** — Carnet de réflexions structuré et accessible partout.

1.2 Le projet KerpheX Blog

KerpheX est un blog personnel professionnel développé avec le framework **Laravel 13**. Il répond à un besoin précis :

- Seul l'**administrateur** peut publier, modifier et supprimer des articles.
- Les **visiteurs** peuvent lire, commenter, liker et ajouter des articles en favoris.
- L'interface est moderne, responsive et entièrement en français.
- Le site fonctionne en local (Laragon) et en production (Railway).

Ce projet démontre la maîtrise d'une stack web complète : backend PHP, base de données, authentification, interface utilisateur, déploiement cloud et bonnes pratiques de développement.

2. Technologies et outils utilisés

Technologie	Rôle dans le projet	Pourquoi ce choix ?
PHP 8.3	Langage serveur	Standard du web, performant, compatible Laragon et Railway
Laravel 13	Framework MVC	Routing, ORM Eloquent, Blade, auth, migrations — productivité maximale
SQLite	Base de données	Fichier unique, zéro configuration, idéal dev local + petit déploiement
Laravel Breeze	Authentification	Scaffolding login/register/reset password prêt à l'emploi
Laravel Socialite	OAuth Google	Connexion rapide via compte Google
Blade	Moteur de templates	Syntaxe PHP simplifiée pour les vues HTML
Vite 7	Build frontend	Compilation rapide CSS/JS pour la production
CSS custom	Design KerpheX	Identité visuelle unique (polices Fraunces + Source Sans 3)
Composer	Gestionnaire PHP	Installe Laravel et ses dépendances
npm / Node.js	Gestionnaire JS	Compile les assets via Vite
Git + GitHub	Versionnement	Historique du code, déclenchement du déploiement
Railway	Hébergement cloud	Déploiement automatique à chaque push GitHub
Nixpacks	Build Railway	Détecte PHP, installe Composer, lance l'app
PHPUnit	Tests	Vérifie que l'authentification et les pages fonctionnent
Laragon	Environnement local	Apache/Nginx + PHP + MySQL/SQLite sur Windows

3. Environnement de développement — Laragon

Laragon est un environnement de développement web tout-en-un pour Windows. Il fournit :

- **Apache** (serveur web) — sert les fichiers PHP
- **PHP 8.3** — interprète le code Laravel
- **Composer** — installe les packages PHP
- **Node.js** — pour Vite et npm
- **Création automatique de virtual hosts** — `http://mon-blog.test`

3.1 Installation locale — étapes réalisées

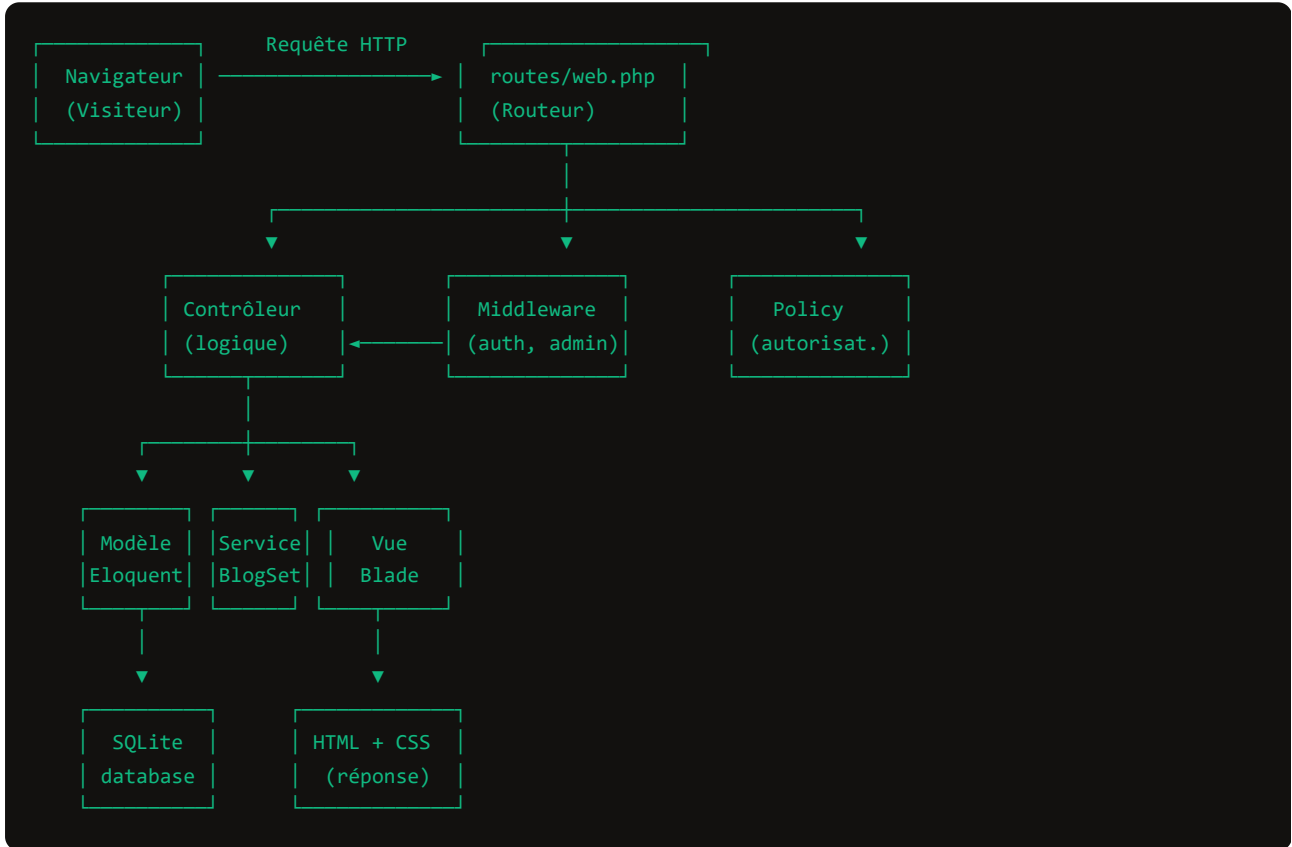
1. Placer le projet dans `C:\laragon\www\mon-blog`
2. `composer install` — télécharge Laravel et dépendances dans `vendor/`
3. `copy .env.example .env` — fichier de configuration locale
4. `php artisan key:generate` — clé de chiffrement unique (`APP_KEY`)
5. Créer `database/database.sqlite` — base SQLite vide
6. `php artisan migrate` — crée les tables (users, posts, comments...)
7. `php artisan db:seed` — insère le compte admin et les articles démo
8. `php artisan storage:link` — lien symbolique pour les images uploadées
9. `npm install && npm run build` — compile CSS/JS dans `public/build/`
10. `php artisan serve` — serveur dev sur `http://127.0.0.1:8000`

Pourquoi `php artisan serve` en plus de Laragon ?

Artisan serve utilise le serveur PHP intégré — pratique pour le développement rapide sans configurer Apache. Laragon reste disponible pour un environnement plus proche de la production.

4. Architecture globale du projet

Laravel suit le pattern **MVC** (Modèle — Vue — Contrôleur) :



Flux simplifié : L'utilisateur visite une URL → Laravel trouve la route correspondante → le middleware vérifie les permissions → le contrôleur exécute la logique (lit/écrit en base) → une vue Blade est rendue en HTML → le navigateur affiche la page.

5. Structure des dossiers — chaque partie expliquée

5.1 Dossier `app/` — Le code métier PHP

Fichier / Dossier	Pourquoi il existe
<code>Http/Controllers/PostController.php</code>	Gère l'accueil, l'affichage, la création, l'édition et la suppression des articles. C'est le contrôleur le plus important du blog.
<code>Http/Controllers/CommentController.php</code>	Permet aux visiteurs connectés de poster des commentaires ; l'admin peut les supprimer.
<code>Http/Controllers/LikeController.php</code>	Ajoute ou retire un « j'aime » sur un article (1 par utilisateur).
<code>Http/Controllers/FavoriteController.php</code>	Gère les favoris : marquer/démarquer un article + page « Mes favoris ».
<code>Http/Controllers/SearchController.php</code>	Recherche dans titre, contenu, catégories et tags.
<code>Http/Controllers/CategoryController.php</code>	Liste et filtre les articles par catégorie (Sport, Santé, Tech...).
<code>Http/Controllers/TagController.php</code>	Affiche le nuage de tags et filtre par mot-clé.
<code>Http/Controllers/PageController.php</code>	Page statique « À propos » avec statistiques du blog.
<code>Http/Controllers/ProfileController.php</code>	Permet au visiteur de modifier son profil ou supprimer son compte.
<code>Http/Controllers/Admin/DashboardController.php</code>	Tableau de bord admin : stats, modération commentaires, liste articles.
<code>Http/Controllers/Admin/AdminLoginController.php</code>	Connexion admin séparée — refuse les comptes visiteur.
<code>Http/Controllers/Admin/UserManagementController.php</code>	Bloquer, débloquer ou supprimer des utilisateurs.
<code>Http/Controllers/Admin/SettingsController.php</code>	Paramètres du blog + export CSV utilisateurs/stats.
<code>Http/Controllers/Admin/MediaController.php</code>	Bibliothèque d'images de couverture (upload, suppression).
<code>Http/Controllers/Auth/*</code>	Contrôleurs Breeze : login, register, reset password, Google OAuth.
<code>Models/User.php</code>	Représente un utilisateur (admin ou visiteur), avec rôle et statut bloqué.
<code>Models/Post.php</code>	Représente un article : slug auto, tags, compteur vues, temps de lecture.

Models/Comment.php	Commentaire avec modération (approved) et réponses imbriquées (parent_id).
Models/Like.php	Enregistre un like user ↔ article (contrainte unique).
Models/Favorite.php	Enregistre un favori user ↔ article.
Policies/PostPolicy.php	Règle : seul l'admin peut créer/modifier/supprimer des articles.
Services/BlogSettings.php	Lit/écrit les paramètres du blog dans un fichier JSON (pagination, modération...).
Middleware/AdminMiddleware.php	Bloque l'accès aux routes /admin/* si l'utilisateur n'est pas admin.
Middleware/CheckBlocked.php	Déconnecte automatiquement un utilisateur bloqué par l'admin.
Providers/AppServiceProvider.php	Force HTTPS en production + enregistre PostPolicy.

5.2 Dossier `routes/`

`web.php` définit toutes les URLs du site (accueil, articles, admin, recherche...). `auth.php` contient les routes d'authentification générées par Breeze (login, register, forgot password). `console.php` définit les commandes Artisan personnalisées.

5.3 Dossier `resources/`

Élément	Rôle
css/app.css	~2000 lignes de CSS custom : variables de couleur, typographie, cartes articles, navigation, admin. Design éditorial unique KerpheX.
js/app.js	Point d'entrée Vite — importe Axios pour requêtes HTTP.
views/layouts/app.blade.php	Layout public : navbar, recherche, menu mobile, footer, barre de progression.
views/layouts/admin.blade.php	Layout panneau d'administration.
views/layouts/auth.blade.php	Layout pages connexion/inscription.
views/components/post-card.blade.php	Carte article réutilisable (image, titre, catégorie, icônes like/commentaire).
views/components/kerphex-logo.blade.php	Logo SVG KerpheX.
views/posts/	Pages accueil, détail article, création et édition.
views/admin/	Toutes les pages du panneau admin.
lang/fr/	Messages d'erreur et validation en français.

5.4 Autres dossiers importants

- `database/migrations/` — Scripts qui créent/modifient les tables SQLite (versionnés, rejouables).
- `database/seeders/` — Données initiales (admin, articles démo).
- `public/index.php` — Point d'entrée unique de toute requête HTTP.
- `public/build/` — CSS/JS compilés par Vite (versionnés pour Railway).
- `storage/app/public/covers/` — Images de couverture des articles.
- `config/` — Configuration Laravel (app, auth, database, blog.php...).
- `tests/` — 25 tests PHPUnit (auth + pages).

6. Base de données et modèles de données

6.1 Pourquoi SQLite ?

SQLite stocke toute la base dans un seul fichier (`database/database.sqlite`). Pas de serveur MySQL à installer — parfait pour Laragon en dev et Railway en production (petit trafic).

6.2 Tables principales

Table	Colonnes clés	Utilité
users	name, email, password, role, blocked	Comptes admin et visiteurs
posts	title, slug, content, category, tags, cover_image, views, published	Articles du blog
comments	post_id, user_id, body, approved, parent_id	Commentaires modérés avec réponses
likes	post_id, user_id (unique)	J'aime par utilisateur
favorites	user_id, post_id (unique)	Articles favoris
sessions	id, user_id, payload	Sessions utilisateur (driver database)
cache	key, value	Cache Laravel (driver database)

6.3 Relations Eloquent

Un **User** possède plusieurs **Posts**. Un **Post** a plusieurs **Comments**, **Likes** et **Favorites**. Les favoris lient User et Post en many-to-many. Les commentaires peuvent avoir un `parent_id` pour les réponses de l'admin.

6.4 Migrations — pourquoi chaque migration existe

- `create_users_table` — Table utilisateurs + sessions + reset password (base Laravel).
- `create_posts_table` — Structure des articles avec slug unique.
- `create_comments_table` — Commentaires liés aux articles.
- `create_likes_table` — Système de likes (initialement par IP, puis par user).
- `add_role_to_users` — Distinction admin / visitor.
- `create_favorites_table` — Système de favoris.
- `add_blocked_to_users` — Permet à l'admin de bloquer un visiteur.
- `add_parent_id_to_comments` — Réponses imbriquées aux commentaires.
- `add_user_id_to_likes` — Likes par utilisateur connecté.
- `add_unique_constraint_to_likes` — 1 seul like par user/article.
- `add_tags_to_posts` — Mots-clés CSV sur les articles.

7. Routes et contrôleurs

7.1 Routes publiques (sans connexion)

URL	Action
/	Page d'accueil — articles publiés, article à la une, catégories
/posts/{slug}	Détail d'un article (incrémente les vues)
/categories	Liste des catégories
/tags	Nuage de tags
/search?q=...	Recherche full-text
/about	Page À propos
/login et /register	Connexion et inscription visiteur
/admin/login	Connexion administrateur (séparée)

7.2 Routes authentifiées visiteur

Commenter, liker, favoriser, gérer son profil — nécessitent une session active (`middleware auth`).

7.3 Routes admin

Préfixe `/admin/*`, protégées par `middleware auth + admin` : dashboard, CRUD articles, modération, utilisateurs, médias, paramètres, exports CSV.

8. Vues Blade et interface utilisateur

Blade est le moteur de templates Laravel. Il permet d'écrire du HTML avec des directives PHP simplifiées :

- `@extends('layouts.app')` — hérite d'un layout parent
- `@section('content')` — définit le contenu injecté dans le layout
- `@auth / @guest` — affiche du contenu selon l'état de connexion
- `{{ $variable }}` — affiche une variable échappée (sécurité XSS)
- `<x-post-card :post="$post" />` — composant Blade réutilisable

Le design KerpheX utilise une palette vert forêt (#0d7a55), fond crème (#f7f4ef), typographie serif Fraunces pour les titres et sans-serif Source Sans 3 pour le corps. L'interface est entièrement responsive (menu hamburger mobile, grilles adaptatives).

9. Authentification et gestion des rôles

9.1 Deux portes d'entrée

Portail	URL	Qui peut entrer ?
Visiteur	<code>/login</code>	Tout le monde (inscription ouverte)
Administrateur	<code>/admin/login</code>	Uniquement les comptes <code>role=admin</code>

9.2 Rôles

- **admin** — Publie des articles, modère, gère les utilisateurs, accède au panneau.
- **visitor** — Lit, commente, like, favorise. Ne peut pas publier.

9.3 Google OAuth

Via Laravel Socialite, un visiteur peut se connecter avec Google. Si son email figure dans `ADMIN_EMAILS` (.env), il est automatiquement promu administrateur.

9.4 Sécurité

- Mots de passe hashés avec **bcrypt** (jamais stockés en clair).
- Protection **CSRF** sur tous les formulaires (`@csrf`).
- Rate limiting sur le login (5 tentatives max).
- Middleware `CheckBlocked` — déconnexion si compte bloqué.
- `PostPolicy` — double vérification avant toute action sur un article.
- HTTPS forcé en production.

10. Fonctionnalités détaillées

10.1 Articles

L'admin crée un article avec titre, extrait, contenu, catégorie, tags et image de couverture. Un **slug** URL-friendly est généré automatiquement (ex: `coupe-du-monde-2026-tout-ce-qu'il-faut-savoir`). L'article peut être publié ou en brouillon. Chaque lecture incrémente le compteur de vues.

10.2 Commentaires

Les visiteurs connectés postent des commentaires. L'admin peut les approuver, rejeter, répondre ou supprimer. L'auto-approbation est configurable dans les paramètres du blog.

10.3 Likes et favoris

Like = réaction rapide (cœur). Favori = marque-page personnel. Les deux nécessitent une connexion. Les icônes sur les cartes articles redirigent vers `/login` si non connecté.

10.4 Recherche, catégories et tags

La recherche parcourt titre, extrait, contenu, catégorie et tags. Les catégories et tags sont des champs texte sur les articles (pas de tables séparées — simplicité volontaire pour un blog de taille modeste).

10.5 Panneau admin

Dashboard avec 9 indicateurs (articles, vues, commentaires en attente, utilisateurs, likes, favoris...). Gestion complète du contenu, des utilisateurs, des médias et export CSV pour analyse.

11. Services, middleware et sécurité

11.1 BlogSettings — pourquoi un fichier JSON ?

Plutôt que de modifier la base de données pour chaque paramètre (nom du blog, pagination, modération auto), `BlogSettings` lit/écrit `storage/app/blog_settings.json`. L'admin modifie ces valeurs depuis le panneau sans toucher au code.

11.2 config/blog.php

Centralise la liste des emails administrateurs (`ADMIN_EMAILS`) utilisée par le seeder et Google OAuth.

12. Assets frontend — CSS, JavaScript, Vite

Vite compile `resources/css/app.css` et `resources/js/app.js` en fichiers optimisés dans `public/build/`. En développement, `npm run dev` recharge automatiquement les changements (HMR). En production, `npm run build` génère des fichiers minifiés avec hash dans le nom (cache busting).

Le layout utilise `@vite(['resources/css/app.css', 'resources/js/app.js'])` pour charger les bons fichiers selon l'environnement.

13. Seeders et données de démonstration

Seeder	Contenu créé	Quand ?
<code>AdminUserSeeder</code>	Compte admin principal	Migration + <code>railway-init.sh</code>
<code>ArticlesSeeder</code>	4 articles : Coupe du Monde, Santé, IA, Wadagni	Si base vide (Railway)
<code>UsersSeeder</code>	3 visiteurs de test (dev local uniquement)	Manuel

14. Tests automatisés

25 tests PHPUnit couvrent : rendu des pages login/register, authentification, reset password, vérification email, profil, et réponse HTTP 200 de l'accueil. Commande : `php artisan test`.

15. Déploiement — GitHub et Railway

15.1 Pipeline de déploiement

```
Développeur (Laragon)
|
├─ npm run build (compile CSS/JS)
├─ git add / commit / push
|
▼
GitHub (mon-blog-laravel)
|
▼ webhook automatique
Railway.app
|
├─ Nixpacks : PHP 8.3 + composer install
├─ Build : cache config/routes/views
├─ Start : railway-init.sh
|   └─ Crée database.sqlite
|       └─ php artisan migrate
|           └─ Seed admin + articles
|               └─ php artisan storage:link
└─ Serveur PHP sur $PORT
|
▼
https://web-production-c5c2f.up.railway.app
```

15.2 Fichiers de déploiement

- `nixpacks.toml` — Indique à Railway comment builder l'app (PHP 8.3, Composer, caches).
- `railway-init.sh` — Script bash exécuté à chaque démarrage du conteneur.
- `Procfile` — Commande alternative de démarrage.
- `public/build/` — Assets versionnés (Railway n'a pas besoin de Node.js).
- `storage/app/public/covers/` — Images de couverture versionnées.

15.3 Variables d'environnement Railway

Variable	Rôle
<code>APP_KEY</code>	Chiffrement sessions et cookies (obligatoire)
<code>APP_URL</code>	URL production pour les liens absolus
<code>APP_ENV=production</code>	Active HTTPS forcé, désactive le debug
<code>DB_CONNECTION=sqlite</code>	Base SQLite embarquée
<code>ADMIN_EMAILS</code>	Emails promus administrateur

16. Liens utiles et identifiants

Environnement	Page	URL
Production	Accueil	https://web-production-c5c2f.up.railway.app
Production	Connexion visiteur	https://web-production-c5c2f.up.railway.app/login
Production	Connexion admin	https://web-production-c5c2f.up.railway.app/admin/login
Local Laragon	Accueil	http://127.0.0.1:8000 ou http://mon-blog.test
GitHub	Code source	https://github.com/kerphileadome-dot/mon-blog-laravel

Compte administrateur :

Email : kerphilesaint@gmail.com

Connexion via `/admin/login`

17. Conclusion

Le projet **KerpheX Blog** est une application web complète qui illustre le cycle de vie d'un produit numérique moderne :

1. **Conception** — Blog éditorial avec rôles admin/visiteur.
2. **Développement** — Laravel 13, PHP 8.3, SQLite, Blade, CSS custom.
3. **Environnement local** — Laragon sur Windows pour développer et tester.
4. **Qualité** — Tests PHPUnit, middleware de sécurité, politiques d'autorisation.
5. **Déploiement** — GitHub + Railway avec pipeline automatisé.
6. **Production** — Site accessible mondialement avec HTTPS.

Chaque fichier du projet a une raison d'exister : les *contrôleurs* traitent les requêtes, les *modèles* représentent les données, les *vues* affichent l'interface, les *migrations* structurent la base, les *seeders* peuplent les données initiales, et les *scripts de déploiement* assurent que le site démarre correctement en production.

Ce document constitue la présentation complète du projet KerpheX, réalisé par Kerphile Saint — Juin 2026.